# DOG BREED CLASSIFICATION USING MULTI-TASK LEARNING

A PREPRINT

Department of Computer Science
Aarhus University

Department of Computer Science
Aarhus University

Department of Computer Science
Aarhus University

December 4, 2025

**ABSTRACT**

This report investigates whether Multi-Task Learning (MTL) can improve dog breed classification by leveraging localization, landmark detection, and segmentation as auxiliary tasks. We train numerous single-task and multi-task deep convolutional neural networks to classify the 12.538 images in the StanfordExtra Dogs Dataset of 120 classes. The best model achieves $76.5\%$ validation accuracy in classification. For the MTL models, we investigate different strategies for aggregating the losses of the different tasks, as model performance is strongly influenced by the weighing of the losses. Initially, we build hard parameter sharing MTL networks, but the gradients of the tasks reveal that none of the auxiliary tasks help the models becoming better classifiers. However, some of the auxiliary task gradients are closely related, and MTL improves performance for these tasks compared to training them individually. Most notably, the best MTL model achieves an intersection over union (IoU) of $82.2\%$ for localization, compared to $63.9\%$ for the baseline. To mitigate the issues with the classification task in MTL, we build soft parameter sharing cross stich networks, they improve the MTL models' performance in classification slightly. The best MTL model achieves $72.4\%$ validation accuracy in classification.

***Keywords*** Image classification · Localization · Landmark prediction · Image segmentation · Multi-Task Learning

## 1 Introduction

Image classification is a fundamental problem in machine learning that captures the potential of models to interpret and understand visual data. Conceptually, the task is straightforward: given an input image and a predefined set of classes, the model must predict which class the image belongs to. Recent advances in deep neural networks demonstrate remarkable performance in visual recognition tasks, in some cases surpassing human-level accuracy He et al. [2015a].

Dog breed classification presents a particularly challenging instance of image classification. Domesticated for millennia, dogs have been selectively bred for a wide range of purposes, resulting in numerous breeds with distinct but often subtle distinguishing features. This high interclass similarity and intraclass variability make distinguishing breeds a non-trivial task Cui et al. [2024]. Accurate breed identification is not only of academic interest but also holds practical importance in veterinary medicine, genetic research, and pet care. Consequently, dog breed classification offers a complex problem for the deep learning and computer vision community.

Despite several models being developed for dog breed classification, which achieve state-of-the-art accuracies exceeding $95\%$ Cui et al. [2024], to the best of our knowledge, the application of Multi-Task Learning (MTL) in this domain remains largely unexplored. This report addresses this gap by investigating the potential benefits of MTL for dog breed classification. We develop a basic classifier consisting of a pretrained ResNet18 encoder He et al. [2015b] and a custom linear output layer. To obtain a baseline model for evaluating later models, we improve the basic model by making the decoder deeper and applying regularization techniques. Finally, we propose several MTL networks capable of simultaneously classifying, predicting landmarks, localizing, and segmentation, in order to evaluate the extent to which MTL can improve performance over traditional single-task models. To evaluate the performance of MTL in the auxiliary tasks, we also build baseline models for these tasks.

## 2 Related Work

Previous research explores a variety of approaches to dog breed classification. Earlier research primarily rely on feature extraction, focusing on key facial characteristics, and applying machine learning techniques such as k-nearest neighbors and support vector machines for classification Chanvichitkul et al. [2007], Liu et al. [2012]. More recent studies primarily rely on CNNs to solve the task, and several approaches have been shown, demonstrating promising result. Some of these approaches focus on building classifiers directly on top of a single CNN Ráduly et al. [2018]. Others combine CNNs with traditional machine learning, as in Cui et al. [2024], where PCA is used for feature extraction on top of a CNN encoder, and Support Vector Machines are used for classification. This approach achieves $95\%$ accuracy on the Stanford Dogs Dataset by Khosla et al. [2011].

**Multi-Task Learning** Multi-Task Learning (MTL) is a technique in which a model is trained for multiple purposes simultaneously Ruder [2017]. This offers several advantages. An obvious advantage is that instead of training separate models for each task, one shared model can be optimized jointly. More importantly, MTL can improve generalization. Even if the main interest is in a single task, adding related auxiliary tasks serves as an inductive bias and regularization. By forcing the network to capture features that are useful across tasks, the model learns richer and more generalizable representations, which can reduce overfitting and help the model focus on the most relevant information Ruder [2017]. For computer vision, MTL usually adopts a hard parameter sharing approach, where the convolutional encoder is shared across all tasks, with a specific decoder for each task handling the final predictions Ruder [2017].

MTL has been used successfully across a wide range of machine learning domains Ruder [2017], particularly in deep learning for visual recognition Chen et al. [2022]. Chen et al. [2022] employs MTL to jointly perform object detection, pose estimation, and image segmentation on fish. The study shows that training a fully end-to-end MTL model not only outperforms training separate models for each task but also surpasses step-by-step approaches, where the network is first trained on one task until it converges and then incrementally fine-tuned with additional tasks.

## 3 Methods

### 3.1 Dataset

The StanfordExtra Dogs Dataset contains 12,568 images of 120 breeds of dogs Biggs et al. [2020]. It is a subset of the Stanford Dogs Dataset Khosla et al. [2011] with additional annotations. Each image is annotated with a class label, a localization target, expressed as a bounding box, landmarks, and a segmentation mask. The landmarks are 20 joints on a dog, but some samples have missing joint annotations. Missing joints are represented as zero or NaN values. When calculating losses, these are excluded. The number of images per breed is unevenly distributed, as shown in Table 1. During training, $80\%$ of the images are used for training and $20\%$ are set aside for validation. The validation samples are used to estimate how well the model generalizes to unseen data. The resolution of the images varies, so part of the preprocessing resizes the images to $224 \times 224$ resolution, which is required for ResNet18 He et al. [2015b].

| No. Samples | Classes | Avg. Count | Min. Count | Max. Count | Std |
|---|---|---|---|---|---|
| 12538 | 120 | 104 | 66 | 183 | 25.66 |

Table 1: Descriptive statistics of the StanfordExtra Dog Dataset

We evaluate classification performance using validation accuracy. For localization and segmentation, we use intersection over union (IoU). For landmark prediction, we use Percentage of Correct Keypoints (PCK) Biggs et al. [2020], which is the percentage of predicted landmarks that is within a certain distance threshold of the true landmark. As a threshold, we use $15\%$ of the square root of the segmentation area, as described in Biggs et al. [2020].

### 3.2 Model training

We train all the following models using the same procedure. We use a pretrained ResNet18 encoder. For the first 10 epochs, we freeze the encoder, enabling the model to adapt to the custom decoder without disrupting the feature extraction capabilities of the encoder learned from a much larger dataset. For MTL models, this allows the task-specific decoders to initially adapt to the shared encoder while it remains fixed. To fine-tune the model for the domain, we unfreeze the last two layers of the shared encoder for the remaining epochs to limit computation time and the risk of overfitting.

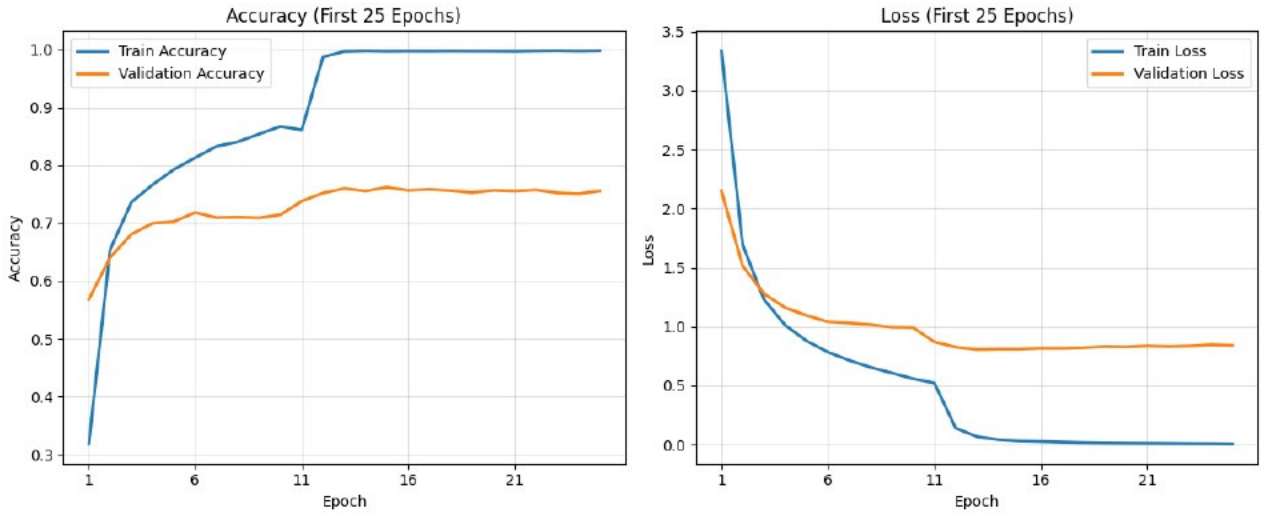



Figure 1: The first 25 epochs of training the basic classifier. We train for 1000 epochs. For the remaining epochs, the validation accuracy and loss worsen, whereas the training accuracy and loss remain at approximately $100\%$ and $0.0$ respectively.

| Top-3 misclassified classes | | | Confused with | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| class | predicted (%) | #samples | class | predicted (%) | #samples | class | predicted (%) | #samples | class | predicted (%) | #samples |
| Lhasa | 0.0 | 66 | Shih-Tzu | 69.23 | 160 | Maltese dog | 15.38 | 183 | Irish wolfhound | 7.69 | 144 |
| Eskimo dog | 8.33 | 67 | Malamute | 33.33 | 121 | Siberian husky | 25.0 | 127 | Dingo | 8.33 | 108 |
| Standard poodle | 33.33 | 86 | Miniature poodle | 14.29 | 102 | Irish water spaniel | 14.29 | 80 | Pug | 4.76 | 108 |

Table 2: An overview of the confusion matrix results for the three most frequently misclassified breeds. On the left, each breed is shown along with the number of samples and the percentage of correct predictions. On the right, the top three classes most often predicted instead are listed, along with the percentage of times each is incorrectly predicted when the breed on the left is the true label, and the number of samples of the class.

Each task requires a separate loss function. Localization and landmark detection are regression problems, so for these, we use the mean squared error (MSE) loss. For classification, we apply the cross-entropy loss. In segmentation, each pixel of the images is labeled as either dog or not a dog, so we use the binary cross-entropy.

For parameter updates, we use the Adam optimizer Kingma and Ba [2017], as it is considered a good default choice, complemented with a scheduler that halves the learning rate when validation accuracy stagnates.

### 3.3 Basic classifier

We build an initial classifier using transfer learning. It consists of a pretrained ResNet18 encoder He et al. [2015b], and as a custom decoder, a fully connected linear layer that takes the $1 \times 1 \times 512$ sized output of the encoder and maps it to a vector with an entry for each of the 120 breeds. Figure 1 shows the training history for this model. During the initial epochs, the model improves on both the training and validation datasets. However, after epoch 10, training accuracy spikes to $100\%$, and the training loss drops to zero within a few iterations, while the model's performance on the validation set only improves marginally, with the best validation accuracy being $75.9\%$. This suggests that the model is overfitting.

#### 3.3.1 Class imbalances

As mentioned in Section 3.1, the number of images per breed is unevenly distributed. To discover if this is an issue for the basic classifier described in Section 3.3, we compute a normalized confusion matrix. Table 2 summarizes the results.

The results in Table 2 reveal that the number of samples of the most misclassified classes are in the lower end in the range of number of samples. Furthermore, these classes tend to be confused with classes containing more samples.

Figure 2: The two images on the left depict samples of the Lhasa breed, while the two images on the right are dogs of the Shih-Tzu breed.

Visual inspection of the dataset shows that samples from the misclassified dog breeds share many visual characteristics with the breeds they are confused with. Figure 2 illustrates this by comparing two samples of the most frequently misclassified breed, Lhasa, with two samples of the breed it is most often mistaken for, Shih-Tzu. The images highlight that the breeds have high intra-class variance, while having low inter-class variance. Meaning that the variance within these two breeds seems to be higher than the variance between the breeds, making them hard to classify. This suggests that the model might learn to predict the class with the most samples to minimize the loss, instead of learning the features that distinguish the two breeds.

### 3.4 Baseline classifier

To address the overfitting of the basic classifier in Section 3.3, we improve it by utilizing several regularization techniques. Specifically, we use data augmentation, dropout, and weight decay.

We use random horizontal flips, rotations, brightness adjustments, crops, and Gaussian noise as augmentations.

As discussed in Section 3.3.1, the class imbalances are an issue for the basic classifier. To handle this, we use a random sampler with weights that ensure that each breed has an equal probability of being selected during training. This means that, on average, the model is presented with the same number of samples per breed during training, diminishing its incentive to predict more frequent breeds.

We also modify the architecture of the decoder. Specifically, we introduce a batch normalization layer and a layer using ReLU activation to introduce non-linearity to the decoder. This enables the model to learn non-linear decision boundaries, which we hypothesize improves the model's capacity to generalize beyond the training data, and hence learn generalizable feature representations within the domain of dog classification. As part of addressing overfitting, we also add a dropout layer before the final linear layer.

Training this model reveals that the changes only delay the point at which the model reaches $100\%$ training accuracy without improving the validation accuracy significantly. The best validation accuracy of the model is $76.5\%$. This shows that the model still overfits and does not learn the feature representations necessary to distinguish the breeds, limiting its performance on unseen data.

Future models are built from this model, and the results of the model serve as a baseline for future experiments.

### 3.5 Multi-Task Learning models

As an attempt to make the model combat overfitting and learn generalizable feature representations, we exploit the additional annotations in the dataset to employ Multi-Task Learning (MTL).

MTL networks can use either hard or soft parameter sharing Ruder [2017]. With hard parameter sharing, all tasks use the same encoder, but separate decoders. In contrast, soft parameter sharing assigns each task its own encoder, but constrains the encoder layers to benefit from one another.

#### 3.5.1 Tasks

Besides classification, we use localization, landmark detection, and segmentation as auxiliary tasks for MTL. Intuitively, these tasks will help classification as they rely on spatial and structural information in the image, supporting the classification task in focusing on the geometry, pose, and spatial location of the dog. Performance for classification can be compared with the model described in Section 3.4. To evaluate whether MTL yields performance gains in the remaining tasks, we construct dedicated baseline models for comparison. To obtain a baseline model for the localization task, we modify the final layer of the model in Section 3.4 such that it predicts 4 values instead of 120. These are the

coordinate pair of the upper left corner, the width, and the height of the localization bounding box. We do the same for the landmark task, but predict 40 values representing a coordinate pair for each of the 20 joints. For the segmentation task baseline model, we use the U-Net architecture Ronneberger et al. [2015] with a ResNet18 encoder. This means that the intermediate feature maps from the ResNet18 layers are used as skip connections to the corresponding decoder layers. We let layer 4 of ResNet18 serve as the bottleneck representation of the network. As decoder blocks, we use the ones presented by Ronneberger et al. [2015], with an additional dropout layer to prevent overfitting.

#### 3.5.2 Hard parameter sharing models

We build a hard parameter sharing MTL model, which we refer to as BasicMTL. We use a shared ResNet18 encoder with task-specific decoders. For classification, we use the same decoder as in Section 3.4, and for the other tasks, the same decoders as in Section 3.5.1. To derive a loss function for backpropagation, we simply sum the different loss functions described in Section 3.2.

In Figure 3, we visualize the training history of the model. After the 10 epochs of frozen training, the classification accuracy drops immediately. A likely explanation is that the task losses differ in scale, implying that tasks with big gradient signal might suppress other tasks in optimizing the shared encoder.

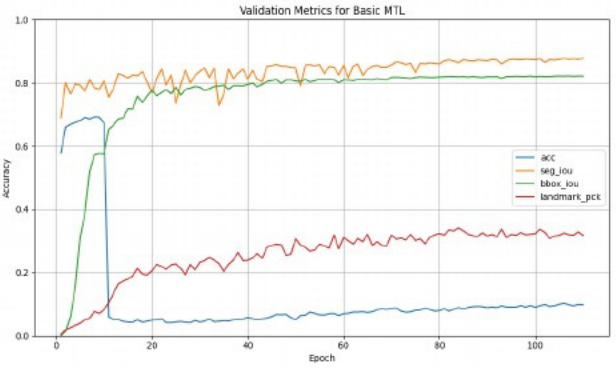



Figure 3: Training history of the BasicMTL model. We train for 110 epochs. The figure clearly shows a drop in classification accuracy immediately after the encoder is unfrozen at epoch 10.

### 3.6 Loss weighting

It is a well-known challenge in MTL that different loss functions may have different scales and gradient magnitudes, leading to imbalanced learning across tasks Yu et al. [2020]. Therefore, the losses of the different tasks are usually weighted to obtain a combined loss. Kendall et al. [2018] shows that the performance of MTL highly depends on the weighing, and some weightings yield worse performance than not employing MTL. Therefore, we explore and build models using the following two strategies to scale the losses.

#### 3.6.1 Grad Norm

The objective of gradient normalization (GradNorm) Yu et al. [2020] is to ensure that all tasks learn at similar rates. To do so, it defines a weight $w_i$ for each task used to scale the loss of task $i$. These task weights $w_i$ are dynamically updated during training, and the aggregated loss is the sum of the individual weighted task losses. During training, the task weights are optimized for the following loss function:

$$L_{grad}(t; w_i(t)) = \sum_{i}^{T} \left| G_W^{(i)}(t) - \bar{G}_W(t) \times [r_i(t)]^{\alpha} \right|$$

where $T$ denotes the number of tasks. $G_W^{(i)}(t)$ denotes the L2 norm of the gradient of the weighted loss of task $i$ at time $t$ with respect to the model parameters shared by the tasks. $\bar{G}_W(t)$ denotes the average $G_W^{(i)}(t)$ for all tasks. $r_i(t)$ represents the relative inverse training rate of task $i$ at time $t$. This value is high for tasks that have learned more slowly than the average task. $\alpha$ serves as a non-linear hyperparameter determining the strength of $r_i(t)$. Without the scaling factor $[r_i(t)]^{\alpha}$, all the gradients of weighted task losses would be equal in the loss optima, but the scaling allows a greater gradient for slow learning tasks, letting them catch up during training. The objective of the loss is to select task weights such that all tasks contribute in a balanced manner to optimizing the shared parameters.

We rebuild the MTL model using GradNorm with a separate Adam optimizer for the task weights. We train the model for 100 epochs, achieving limited results on all tasks except localization, which performs significantly better than its baseline as seen in Table 3.

#### 3.6.2 Uncertainty

Kendall et al. [2018] proposes to weigh the individual losses by the homoscedastic uncertainty of the corresponding task. To obtain a measure of the uncertainty for a regression task, Kendall et al. [2018] defines the likelihood as a

Gaussian with the model's prediction as mean and variance $\sigma^2$. The variance captures the task uncertainty, reflecting that a model's predictions deviates more from the true labels for tasks with a high degree of uncertainty than for tasks with less uncertainty.

For a classification task, the likelihood is defined as the softmax of the model logits scaled by $\frac{1}{\sigma^2}$. To understand how this captures the uncertainty, consider training a model on a task with high uncertainty. Even after updating the model parameters, the loss may remain high, i.e. the logit corresponding to the true label has a low value compared to the other logits, meaning that the softmax function assigns the true label a low probability. Then, increasing the temperature of the softmax would flatten the probability distribution, thus increasing the probability of the true label. By definition of the scaling, $\sigma^2$ corresponds to the temperature, so if assigning a high probability to the true label requires a great $\sigma^2$ value, then the task uncertainty is high.

Consider a MTL network with a regression task and classification task, then by the above definitions Kendall et al. [2018] derives a combined loss with the task uncertainties as learnable parameters $\sigma_1$ and $\sigma_2$:

$$\mathcal{L}(W, \sigma_1, \sigma_2) = \frac{1}{2\sigma_1^2}\mathcal{L}_1(W) + \frac{1}{\sigma_2^2}\mathcal{L}_2(W) + \log \sigma_1 + \log \sigma_2$$

where $\mathcal{L}_1(W)$ is the Euclidean loss of the regression task and $\mathcal{L}_2(W)$ is the cross-entropy loss of the classification task. This approach extends to more than two tasks by adding more terms.

The assumption is that difficult tasks will have larger uncertainty values $\sigma^2$, meaning that their corresponding loss terms are down-weighted through the $\frac{1}{\sigma^2}$ and $\frac{1}{2\sigma^2}$ factors. Conversely, tasks with lower uncertainty receive larger effective gradients, thus contributing more to parameter updating than with a combined loss with no scaling. This formulation allows the model to automatically balance task learning based on their difficulty, hopefully preventing more difficult tasks from dominating how the shared parameters are updated.

We rebuild the MTL model with this loss function. We train for 100 epochs, again achieving limited results on all tasks except localization, which performs significantly better than its baseline as seen in Table 3.

### 3.6.3 Orthogonal gradients

Ruder [2017] argues that hard parameter sharing significantly reduces the risk of overfitting, as the model has to learn features that generalize across all tasks. However, it implicitly assumes that the optimal features for all tasks are closely aligned in the shared feature space. If this is not the case, it could explain the poor performance of the model.

As illustrated in Figure 4, the gradients of most task pairs in the shared encoder are close to orthogonal after 45 epochs, with the exception of localization and segmentation. This suggests that the assumption of nearby optimal features does not hold. Additionally, when the cosine similarity between classification and the auxiliary tasks fluctuates around zero, it suggests that, in the direction defined by the classification gradient, the auxiliary gradients behave effectively as random noise. Their contribution towards classification is therefore negligible. At the same time, it is important to emphasize that the auxiliary gradients are far from random in the shared parameter space, meaning that the auxiliary tasks may divert the learned features towards representations unrelated for classification, hence making training for classification slower and harder.

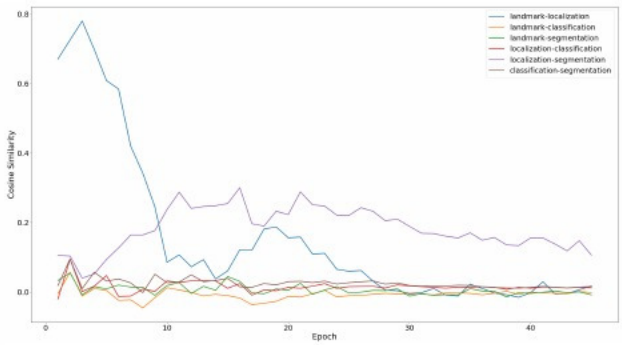



Figure 4: Cosine similarity between gradients of task pairs for GradNormMTL. The figure shows consistent similarity between localization and segmentation, while the similarities among the remaining tasks converge towards zero. This is done for the GradNormMTL model, but it generalizes to all the hard parameter sharing models.

### 3.6.4 Soft parameter sharing model

To address the issues described in Section 3.6.3, we build a new MTL model using the Cross-stitch architecture Misra et al. [2016] and Resnet18 encoders. We insert cross-stitch units after selected layers in the encoders, allowing the model to learn a linear combination of how much knowledge should be shared among tasks. Concretely, let $\boldsymbol{x}_i$ be the feature map of tasks at the i'th layer of the shared network. We introduce a cross-stitch matrix $\mathbf{A}_i$ of trainable

parameters, which will learn a linear combination of the feature maps of shared layer i for all tasks

$$\tilde{\boldsymbol{x}}_{\boldsymbol{i}} = \mathbf{A}_i \boldsymbol{x}_{\boldsymbol{i}}$$

This allows the model to learn an optimal feature-sharing strategy by adjusting the parameters of $\mathbf{A}$ during training. Misra et al. [2016] suggests that cross-stitch units are placed after each pooling operation in the shared network. However, since our shared encoder is based on ResNet18, which contains only two pooling operations, we insert cross-stitch units after the unfrozen layers 3 and 4 as illustrated in Figure 5.

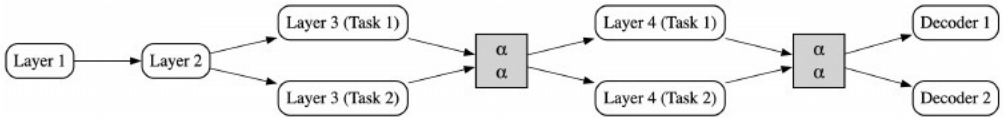



Figure 5: Model of our cross-stitch network architecture, based on ResNet18. In our network, we insert cross-stitch units between layer 3 and layer 4, and again between layer 4 and the task decoder heads. The layers here refer to the layers of ResNet18. This illustration shows the configuration of two tasks. In our implementation, we extend the network to include all 4 tasks

We train this model with both a GradNorm and uncertainty loss weighting. The GradNorm model achieves $72.4\%$ validation accuracy, whereas the uncertainty model achieves $72.3\%$ validation accuracy.

## 4 Results

The results of all the models built in this report are summarized in Table 3.

| | Cls (Acc) | Localization (IoU) | Landmarks (PCK@0.15) | Seg (IoU) |
|---|---|---|---|---|
| BasicCls | 75.9 | - | - | - |
| BaselineCls | 76.5 | - | - | - |
| Localization | - | 63.9 | - | - |
| Landmarks | - | - | 32.8 | - |
| Segmentation | - | - | - | 89.4 |
| BasicMTL | 69.4 | 82.2 | 34.1 | 87.7 |
| GradNormMTL | 70 | 81.5 | 22.6 | 89 |
| UncertaintyMTL | 67.4 | 79.5 | 27.9 | 89.8 |
| CrossStitchGradNormMTL | 72.4 | 78.1 | 24.2 | 90.0 |
| CrossStitchUncertaintyMTL | 72.3 | 81.7 | 28.4 | 89.0 |

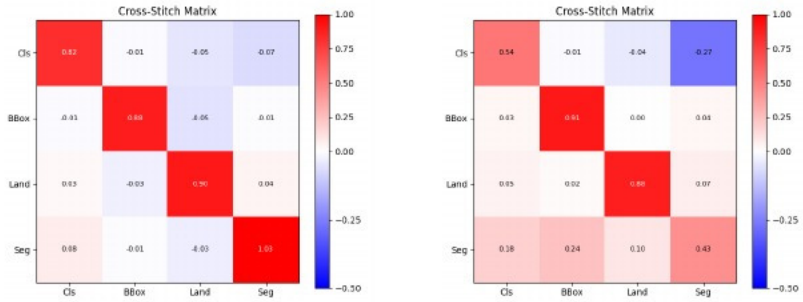
Table 3: The best results of the presented models. BasicCls, BaselineCls, Localization, and Segmentation are trained for 1000 epochs. The MTL models are trained for 100 epochs due to limited computing resources.

The best performance in the classification task is achieved by the classifier presented in Section 3.4. This model achieves $76.5\%$ validation accuracy. All MTL models perform worse, with the best MTL model achieving a validation accuracy of $72.4\%$. However, for other tasks, the MTL models perform better than the corresponding baseline model. For localization, the baseline model achieves $63.9\%$ IoU while the best MTL model, BasicMTL, achieves $82.2\%$ IoU. Similarly, both the landmark and segmentation tasks see slight improvement for some MTL models. Landmark detection reaches a PCK@0.15 of $34.1\%$ compared to the baseline results of $32.8\%$. Segmentation achieves an IoU of $90\%$, slightly above the baseline result of $89.4\%$

We investigate the results of our cross-stitch models by exploring the cross-stitch matrices of the CrossStitchGradNormMTL model. Figure 6 visualizes the matrices. We note that for both the third and fourth layer, the auxiliary tasks have negative cross-stitch weights indicating that the auxiliary tasks do not provide any beneficial feature representations for the classification task. However, for the segmentation task, we observe positive use of the auxiliary tasks, indicating that it benefits from the other tasks.

To further examine the classification results, we visualize which parts of images the model emphasizes when classifying by applying Grad-CAM Selvaraju et al. [2019] on the final convolutional layer of CrossStitchGradNormMTL.

Figure 7 reveals that the most important features used to classify the breed of the top-5 dogs with the lowest loss contain much of the body of the dog, and especially, facial features are used. In contrast, for the top-5 dogs with the highest loss,




(a) Layer 3 cross-stitch matrix (b) Layer 4 cross-stitch matrix

Figure 6: Cross-stitch matrices for the third and fourth layers of the CrossStitchGradNormMTL model, after 100 epochs. In layer 3, the tasks make minimal use of each other when forming their feature representation. In layer 4, we note that segmentation notably benefits from the other tasks, while classification looks to be negatively influenced by the other tasks.

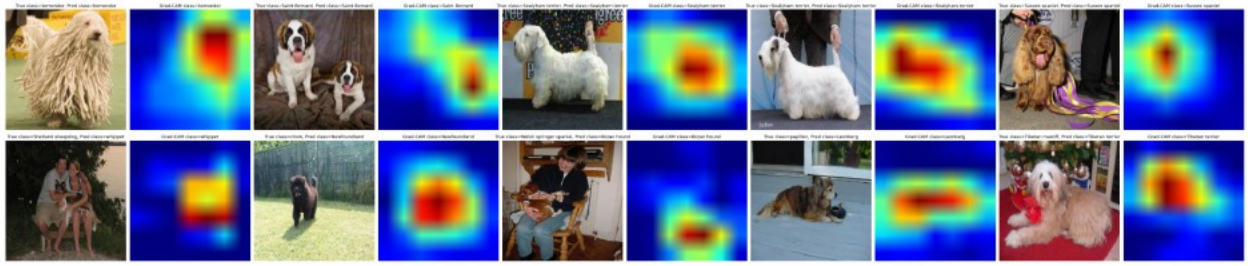


Figure 7: Grad-CAM heat maps of the areas of images emphasized by the CrossStitchGradNormMTL model when classifying the breed. The top row is the top-5 dogs from the validation set with the lowest loss, and the bottom row is the top-5 dogs with the highest loss.

the heat maps show misplaced activations, often focusing on humans, other background elements, or non-distinguishing features. This suggests that the model has not learned a generalizable way to extract important features.

## 5 Discussion

Our results indicate that Multi-Task Learning (MTL) using localization, landmark detection, and segmentation as auxiliary tasks does not improve dog breed classification. However, localization improves significantly using MTL. One possible explanation for the limited performance of the MTL models in classification is the short training duration of 100 epochs. Another reason is that the auxiliary tasks do not benefit the classification task. Our analysis of the cross-stitch matrices and the gradient similarities suggests that the auxiliary tasks do not provide a useful gradient signal, hindering improvement in classification.

The idea behind using MTL is that optimizing for multiple tasks will help the model learn generalizable feature representations, enabling the model to correctly classify unseen images. Ideally, given an unseen image of a dog, the model should learn which features to look for to recognize the breed. However, as seen in Figure 7, this is not always achieved. For the images with the highest classification losses, the network focuses on irrelevant features, so the assumption that the auxiliary tasks will help guide the network towards important features of dogs does not always hold in practice.

For future work, it should be investigated whether training our MTL models for more epochs would give better performance. Additionally, examine how a larger, more state-of-the-art model performs across the various tasks, particularly to assess whether it can match or surpass the performance gains we achieve for some tasks using MTL. Furthermore, exploring how such a model performs using the MTL techniques employed in this project.

## References


Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification, 2015a. URL https://arxiv.org/abs/1502.01852.

Ying Cui, Bixia Tang, Gangao Wu, Lun Li, Xin Zhang, Zhenglin Du, and Wenming Zhao. Classification of dog breeds using convolutional neural network models and support vector machine. *Bioengineering*, 11(11), 2024. ISSN 2306-5354. doi:10.3390/bioengineering11111157. URL https://www.mdpi.com/2306-5354/11/11/1157.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition, 2015b. URL https://arxiv.org/abs/1512.03385.

Massinee Chanvichitkul, Pinit Kumhom, and Kosin Chamnongthai. Face recognition based dog breed classification using coarse-to-fine concept and pca. pages 25–29, 11 2007. ISBN 978-1-4244-1374-4. doi:10.1109/APCC.2007.4433495.

J. Liu, A. Kanazawa, D. Jacobs, and P. Belhumeur. Dog breed classification using part localization. In Andrew Fitzgibbon, Svetlana Lazebnik, Pietro Perona, Yoichi Sato, and Cordelia Schmid, editors, *Proceedings of the 12th European Conference on Computer Vision (ECCV)*, volume 7573 of *Lecture Notes in Computer Science*, pages 172–185, Florence, Italy, October 2012. Springer. doi:10.1007/978-3-642-33718-5_13.

Zalán Ráduly, Csaba Sulyok, Zsolt Vadászi, and Attila Zölde. Dog breed identification using deep learning. In *2018 IEEE 16th International Symposium on Intelligent Systems and Informatics (SISY)*, pages 000271–000276, 2018. doi:10.1109/SISY.2018.8524715.

Aditya Khosla, Nityananda Jayadevaprakash, Bangpeng Yao, and Li Fei-Fei. Novel dataset for fine-grained image categorization. In *First Workshop on Fine-Grained Visual Categorization, IEEE Conference on Computer Vision and Pattern Recognition*, Colorado Springs, CO, June 2011.

Sebastian Ruder. An overview of multi-task learning in deep neural networks. *CoRR*, abs/1706.05098, 2017. URL http://arxiv.org/abs/1706.05098.

Ziwen Chen, Lijie Cao, Qihua Wang, and Yu Cai. Fishnet: Fish visual recognition with one stage multi-task learning. *IET Image Processing*, 16, 06 2022. doi:10.1049/ipr2.12556.

Benjamin Biggs, Oliver Boyne, James Charles, Andrew Fitzgibbon, and Roberto Cipolla. Who left the dogs out: 3D animal reconstruction with expectation maximization in the loop. In *ECCV*, 2020.

Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization, 2017. URL https://arxiv.org/abs/1412.6980.

Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation, 2015. URL https://arxiv.org/abs/1505.04597.

Tianhe Yu, Saurabh Kumar, Abhishek Gupta, Sergey Levine, Karol Hausman, and Chelsea Finn. Gradient surgery for multi-task learning, 2020. URL https://arxiv.org/abs/2001.06782.

Alex Kendall, Yarin Gal, and Roberto Cipolla. Multi-task learning using uncertainty to weigh losses for scene geometry and semantics, 2018. URL https://arxiv.org/abs/1705.07115.

Ishan Misra, Abhinav Shrivastava, Abhinav Gupta, and Martial Hebert. Cross-stitch networks for multi-task learning, 2016. URL https://arxiv.org/abs/1604.03539.

Ramprasaath R. Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra. Grad-cam: Visual explanations from deep networks via gradient-based localization. *International Journal of Computer Vision*, 128(2):336–359, October 2019. ISSN 1573-1405. doi:10.1007/s11263-019-01228-7. URL http://dx.doi.org/10.1007/s11263-019-01228-7.